DAY 3 • SESSION 1d • DATA STRUCTURES & ALGORITHMS

Dynamic Programming

Solving complex problems by breaking them into overlapping subproblems — taught with Java

Foundation Training Programme

90-minute guided class • Concepts + Live Code + Student Workbook

Chitkara × Infosys

Session Agenda & Learning Outcomes

ROADMAP

⏱ 3 min

1	Why DP? The cost of naive recursion	~10 min
2	What DP is + its two defining properties	~14 min
3	Memoization vs Tabulation (with Fibonacci)	~14 min
4	The 5-step DP framework	~4 min
5	Worked problems: Stairs, Coin Change, Knapsack, LCS, LIS	~34 min
6	Patterns, complexity & pitfalls	~6 min
7	Student Workbook: checks + coding exercises	~14 min
8	Recap & Q&A	~5 min

YOU WILL BE ABLE TO

- ✓ Spot a DP problem
- ✓ Write memoized code
- ✓ Build DP tables
- ✓ Analyse cost
- ✓ Solve 5 classics

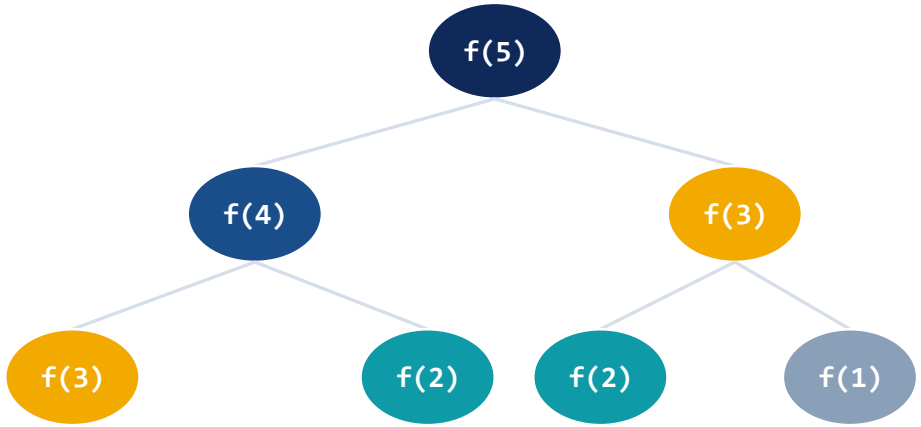
Warm-up: The Hidden Cost of Naive Recursion

5 min

MOTIVATION

Computing Fibonacci with plain recursion recomputes the same values again and again.

```java
int fib(int n) {
  if (n <= 1) return n;
  return fib(n-1) + fib(n-2);
}
```



The problem:
This runs in
$O(2^n)$
time. fib(50) triggers over a billion calls — seconds to minutes of wasted computation on duplicate subproblems.

DP fixes exactly this: compute each unique subproblem once, remember it, reuse it.

What Is Dynamic Programming?

4 min

DEFINITION

Dynamic Programming

is a method for solving a complex problem by breaking it into simpler **overlapping subproblems**, solving each once, and **storing**

Reuse, don't recompute

Each subproblem is solved a single time; its answer is cached and looked up later.

Combine subsolutions

The optimal answer is assembled from the optimal answers of its parts.

Exponential → Polynomial

Adding memory turns many $O(2^n)$ recursions into $O(n)$ or $O(n^2)$.

DP ≠ a specific algorithm — it is a way of thinking about problems that have structure worth exploiting.

The Two Signatures of a DP Problem

CORE THEORY

⏱ 5 min

A problem is a DP candidate only if BOTH hold. Learn to test for them quickly.

1. Optimal Substructure

- The optimal solution of the whole is built from optimal solutions of its parts.
- Fibonacci: fib(n) = fib(n-1) + fib(n-2).
- Shortest path: the best route A→C through B uses the best A→B and B→C.
- **Test: can I write a recurrence relation for it?**

2. Overlapping Subproblems

- The same subproblems recur many times as you expand the recursion.
- Fibonacci: fib(3), fib(2) evaluated over and over.
- This is what makes caching pay off.
- **Test: does the recursion tree repeat nodes?**

Two Styles: Memoization vs Tabulation

APPROACH

⏱ 5 min

	Memoization (Top-Down)	Tabulation (Bottom-Up)
Idea	Recursion + a cache	Iteration filling a table
Direction	Start at the goal, recurse down	Start at base cases, build up
Storage	HashMap / array, filled lazily	Array/2-D table, filled fully
Pros	Natural, only needed states	No recursion/stack overflow, fast
Cons	Call-stack overhead	May compute unused states

Rule of thumb:
Both give the same answer and same big-O time. Start with memoization (easier to derive), switch to tabulation for speed or to dodge deep recursion.

Fibonacci in Three Ways — (1) Naive

4 min

LIVE CODE 1/3

Java

```java
// Exponential time — no memory
static int fib(int n) {
    if (n <= 1)
        return n;              // base case
    return fib(n - 1)
         + fib(n - 2);         // recompute!
}
```

Complexity

Time:
$O(2^n)$

Space:
O(n) call stack

Verdict:
correct but unusable for n > ~40.

Why keep this slide?

This is our baseline. Every optimisation that follows is measured against it. Watch how one line — a cache lookup — collapses $O(2^n)$ into O(n).

Fibonacci in Three Ways — (2) Memoization

LIVE CODE 2/3

⏱ 5 min

Java

```java
static int fib(int n, int[] memo) {
    if (n <= 1) return n;
    if (memo[n] != 0)          // already solved?
        return memo[n];        // reuse cached value
    memo[n] = fib(n-1, memo)
            + fib(n-2, memo);  // store on the way
    return memo[n];
}
// caller: fib(n, new int[n + 1]);
```

Top-Down in one idea

- memo[] caches each answer.
- Check cache before recursing.
- Each state computed once.
- **Time: O(n)**
- **Space: O(n) + stack**

From a billion calls to about n calls — same function, one cache.

Fibonacci in Three Ways — (3) Tabulation

LIVE CODE 3/3

5 min

```java
static int fib(int n) {
    if (n <= 1) return n;
    int[] dp = new int[n + 1];
    dp[0] = 0; dp[1] = 1;              // base cases
    for (int i = 2; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];    // build up
    return dp[n];
}
```

```java
// Space-optimised: O(1) memory
static int fib(int n) {
    int a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        int c = a + b; a = b; b = c;
    }
    return n <= 1 ? n : b;
}
```

dp table fills left → right

Bottom-Up wins

- No recursion → no stack overflow
- Table version: O(n) time, O(n) space
- **Rolling version: O(n) time, O(1) space**
- Fastest, most memory-lean

The 5-Step DP Framework

METHOD

⏱ 4 min

A repeatable recipe you can apply to every DP problem in this class and in interviews.

1	**Define the state**	What does dp[i] (or dp[i][j]) MEAN? Write it in one English sentence.
2	**Write the recurrence**	How is dp[i] built from smaller states? This is the heart of the solution.
3	**Set the base cases**	The smallest inputs whose answers you know directly.
4	**Decide the order**	Top-down (memo) or bottom-up (fill table) — ensure dependencies are ready.
5	**Read off the answer**	Which cell holds the final result? Return it.

Steps 1 & 2 are 90% of the difficulty. Nail the state and recurrence and the code writes itself.

Worked Problem 1 — Climbing Stairs

5 min

PRACTICE

Problem:
You climb a staircase of n steps, taking 1 or 2 steps at a time. How many distinct ways to reach the top?

- **State: dp[i] = ways to reach step i**
- **Recurrence: dp[i] = dp[i-1] + dp[i-2]**
  - Reach i either from i-1 (1 step) or i-2 (2 steps).
- **Base: dp[0]=1, dp[1]=1**
- **Answer: dp[n]**
- **It's Fibonacci in disguise! Same recurrence.**

Java

```java
static int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;          // ways to 1, 2
    for (int i = 3; i <= n; i++) {
        int c = a + b; a = b; b = c;
    }
    return b;
}
```

n=4 → 5 ways:
1+1+1+1, 1+1+2, 1+2+1, 2+1+1, 2+2

Worked Problem 2 — Coin Change (Min Coins)

6 min

PRACTICE

Problem:
Given coin denominations and an amount, find the fewest coins that sum to amount (or -1 if impossible).

Java

```java
static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);   // "infinity"
    dp[0] = 0;                     // 0 coins for 0
    for (int a = 1; a <= amount; a++)
        for (int coin : coins)
            if (coin <= a)
                dp[a] = Math.min(dp[a],
                                 dp[a - coin] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

- **State: dp[a] = min coins to make amount a**
- **Recurrence:**
  - dp[a] = min(dp[a-coin] + 1) over coins
- **Base: dp[0] = 0**
- **Answer: dp[amount]**
- **Time O(amount × coins), Space O(amount)**

Worked Problem 3 — 0/1 Knapsack (Setup)

PRACTICE

⏱ 4 min

Problem:
Given items each with a weight and value, and a bag of capacity W, choose a subset with maximum total value without exceeding W. Each item is taken (1) or left (0) — no fractions.

Example items (capacity W = 5)

Item	Weight	Value
A	1	1
B	3	4
C	4	5
D	5	7

Think before we code

- For each item you make a binary choice: take it or skip it.
- Taking an item consumes capacity but adds value.
- **State needs TWO dimensions: which items considered, and capacity left.**
- **dp[i][w] = best value using first i items within capacity w**
- **Try the example by hand first — we prove the optimum with the table next slide.**

0/1 Knapsack — Recurrence, Table & Code

7 min

PRACTICE

Recurrence:

```
dp[i][w] = max( dp[i-1][w]
  (skip item i),
val[i] + dp[i-1][w - wt[i]]
  (take item i) )
```

Java

```java
static int knapsack(int W, int[] wt, int[] val) {
    int n = wt.length;
    int[][] dp = new int[n + 1][W + 1];
    for (int i = 1; i <= n; i++)
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i-1][w];            // skip
            if (wt[i-1] <= w)                  // can take?
                dp[i][w] = Math.max(dp[i][w],
                    val[i-1] + dp[i-1][w - wt[i-1]]);
        }
    return dp[n][W];
}
```

- **State: dp[i][w] = best value, first i items, capacity w**
- **Two choices per cell:**
  - Skip → inherit dp[i-1][w]
  - Take → val + dp[i-1][w-wt]
- **Base: dp[0][*] = 0 (no items)**
- **Answer: dp[n][W]**
- **Time & Space: O(n × W)**

Longest Common Subsequence (LCS)

PRACTICE · PROBLEM 4

⏱ 6 min

Problem:
Given two strings, find the length of the longest subsequence common to both (characters in order, not necessarily contiguous).

Java

```java
static int lcs(String a, String b) {
    int m = a.length(), n = b.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            if (a.charAt(i-1) == b.charAt(j-1))
                dp[i][j] = dp[i-1][j-1] + 1;   // match
            else
                dp[i][j] = Math.max(dp[i-1][j],
                                    dp[i][j-1]);
    return dp[m][n];
}
```

- **State: dp[i][j] = LCS of a[0..i) and b[0..j)**
- **Match → 1 + diagonal dp[i-1][j-1]**
- **No match → max(top, left)**
- **Base: empty string → 0**
- **Time & Space: O(m × n)**

Example:
"ABCBDAB" & "BDCAB" → LCS length 4 (e.g. "BCAB"). Powers file-diff, git, DNA alignment.

Longest Increasing Subsequence (LIS)

PRACTICE · PROBLEM 5

⏱ 4 min

Problem:
Find the length of the longest strictly increasing subsequence in an array of integers.

Java

```java
static int lis(int[] nums) {
    int n = nums.length, best = 1;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);            // each item alone
    for (int i = 1; i < n; i++)
        for (int j = 0; j < i; j++)
            if (nums[j] < nums[i])
                dp[i] = Math.max(dp[i], dp[j] + 1);
    for (int v : dp) best = Math.max(best, v);
    return best;
}
```

- **State: dp[i] = LIS length ENDING at i**
- **Recurrence: dp[i] = max(dp[j]+1) for j<i where nums[j]<nums[i]**
- **Base: dp[i] = 1 (element by itself)**
- **Answer: max of all dp[i]**
- **Time $O(n^2)$, Space O(n)**

Example:
[10, 9, 2, 5, 3, 7, 101, 18] → LIS length 4 (2, 3, 7, 18 or 2, 3, 7, 101).
Note: an O(n log n) patience-sorting version also exists.

Recognising DP Patterns

SYNTHESIS

⏱ 3 min

Most interview DP problems map to a handful of recurring shapes. Learn the shapes, not just the problems.

Linear / 1-D

dp[i] from dp[i-1], dp[i-2]

Fibonacci, Climbing Stairs, House Robber

Unbounded choice

reuse items freely

Coin Change, Rod Cutting

0/1 subset (2-D)

take-or-skip over items

0/1 Knapsack, Subset Sum, Partition

Two sequences (grid)

dp[i][j] over two inputs

LCS, Edit Distance, Common Substring

Ending-at-index

dp[i] = best chain ending at i

LIS, Max Subarray

Interval DP

dp[i][j] over a range

Matrix Chain, Palindrome Partition

Complexity & Common Pitfalls

RIGOUR

⏱ 3 min

How to compute DP complexity

Time = (number of states) × (work per state)

```
Fibonacci:  n states × O(1)  =  O(n)

Knapsack:  n×W states × O(1)  =  O(nW)

LCS:  m×n states × O(1)  =  O(mn)

LIS:  n states × O(n)  =  O(n²)
```

Space can often be reduced by keeping only the rows/values a transition actually reads.

Top pitfalls to avoid

- Wrong / vague state definition — fix step 1 first.
- Off-by-one on array sizes (use n+1).
- Wrong fill order — a state read before it is ready.
- Bad sentinel (0 vs -1) causing false cache hits.
- Reusing the same row in 0/1 problems (turns it unbounded).
- Forgetting the final answer may not be in the last cell (LIS).

Student Workbook — Part A: Concept Checks

WORKBOOK 1/3

⏱ 4 min

Answer individually in your workbook (2 min), then compare with your neighbour (2 min).

Q1. Name the two properties a problem must have to be solvable by DP.

Q2. In one sentence each, distinguish memoization from tabulation.

Q3. What is the time complexity of naive recursive Fibonacci, and why?

Q4. For 0/1 Knapsack, why does the recurrence read the PREVIOUS row (i-1)?

Q5. Give the general formula for the time complexity of a DP solution.

Q6. Why can greedy fail on Coin Change but DP always succeeds?

Student Workbook — Part B: Coding Exercises

WORKBOOK 2/3

⏱ 6 min

Pick ONE and implement it in Java using the 5-step framework. Skeletons provided — fill the logic.

Exercise 1 — House Robber

Max sum of non-adjacent numbers in an array.

Java

```java
static int rob(int[] nums) {
    int n = nums.length;
    if (n == 0) return 0;
    int[] dp = new int[n + 1];
    dp[1] = nums[0];
    for (int i = 2; i <= n; i++)
        // TODO: choose skip vs rob i
        dp[i] = Math.max(
            dp[i-1],
            dp[i-2] + nums[i-1]);
    return dp[n];
}
```

Exercise 2 — Unique Grid Paths

Ways to go top-left → bottom-right moving only right/down.

Java

```java
static int uniquePaths(int m, int n) {
    int[][] dp = new int[m][n];
    for (int i = 0; i < m; i++)
        dp[i][0] = 1;          // first column
    for (int j = 0; j < n; j++)
        dp[0][j] = 1;          // first row
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            // TODO: from top + from left
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
    return dp[m-1][n-1];
}
```

Workbook C — Challenge & Take-Home

WORKBOOK 3/3

⏱ 4 min

In-Class Challenge (attempt now)

- **Edit Distance: min insert/delete/replace to turn word A into word B.**
  - Hint: dp[i][j] over the two words; 3-way min on mismatch.
- **Subset Sum: can any subset hit a target T?**
  - Hint: boolean dp[i][t], take-or-skip like 0/1 Knapsack.

Take-Home Set (before next class)

- Rewrite ONE of today's problems in the OTHER style (memo ↔ tab).
- Coin Change II: count the number of ways (not min coins).
- Longest Palindromic Subsequence (hint: LCS of s and reverse(s)).
- Reduce House Robber to O(1) space.
- **Write, in words, the state + recurrence for each before coding.**

Recap & Key Takeaways

- **DP = solve overlapping subproblems once, store, reuse.**
- Two must-have properties: optimal substructure + overlapping subproblems.
- Two styles: memoization (top-down) and tabulation (bottom-up) — same big-O.
- Follow the 5 steps: state → recurrence → base → order → answer.
- Time ≈ states × work per state; optimise space by dropping unused history.
- **You can now solve Fibonacci, Stairs, Coin Change, Knapsack, LCS, LIS.**

REMEMBER

"Write the state in one English sentence before you write any code."

Next session:
Greedy vs DP — when each wins.

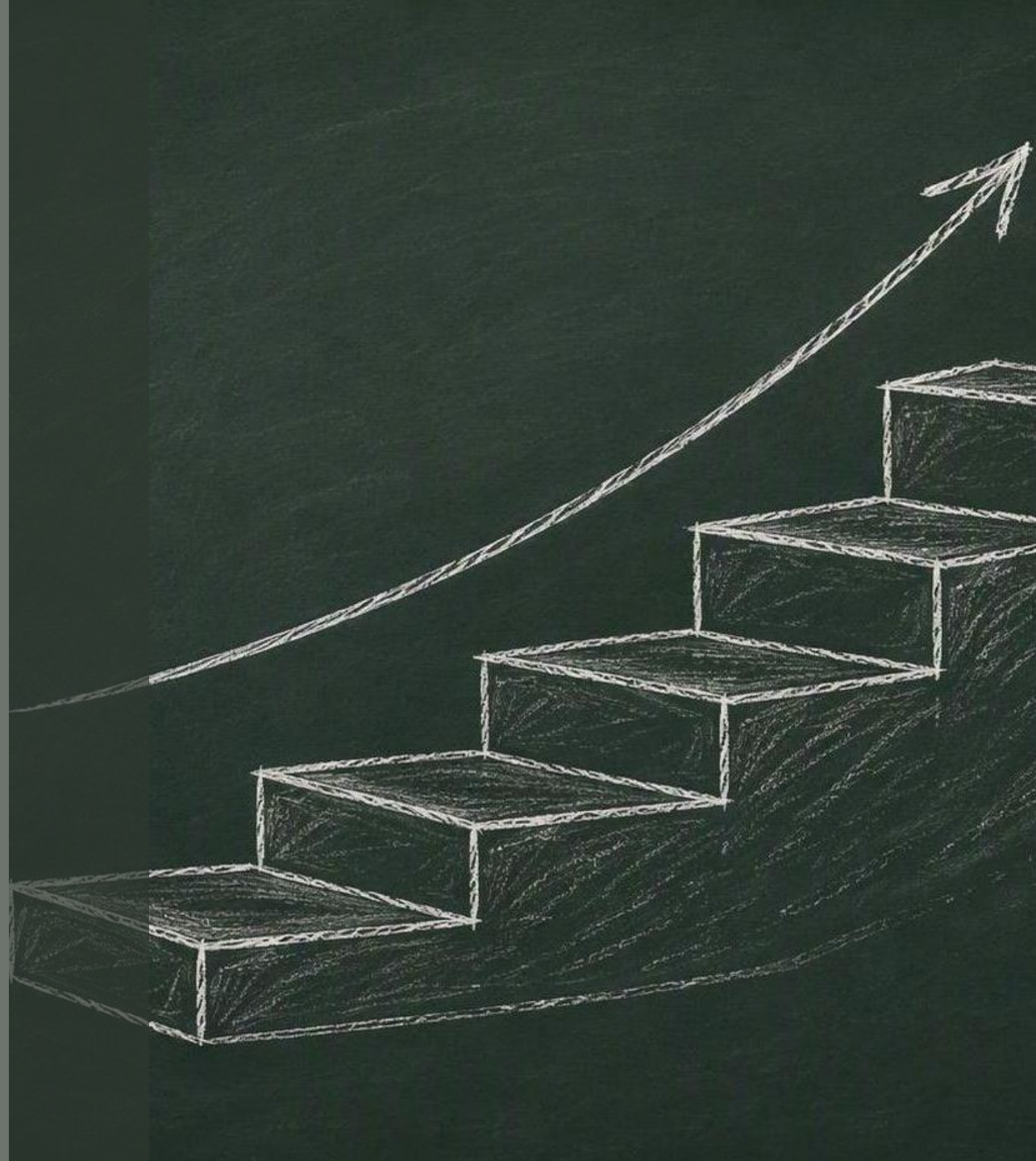
Questions? Let's discuss.

DAY 3 · CLASS CONCLUSION

1D Linear Dynamic Programming

Recap & review session — from recursion to DP thinking.

Structured DP progression · 12-Day Plan



Day 3 Overview: 1D Linear DP

WHERE WE ARE

The Goal

- First real Dynamic Programming day
- Move from recursion → DP thinking

Focus Skills

- Define the DP state
- Build the recurrence
- Solve it iteratively

Core Pattern

- Linear dependency
- previous index → current index

Takeaway: *each answer is built directly from the answer just before it.*

DP Pattern Recognition

WHEN DOES 1D DP APPLY?

Use 1D DP when…

- The problem is sequential
- The answer depends on previous states
- The same work is computed again and again

Common signals in the wording

“Number of ways to reach…”

“Minimum / maximum cost to…”

“Jump from previous steps…”

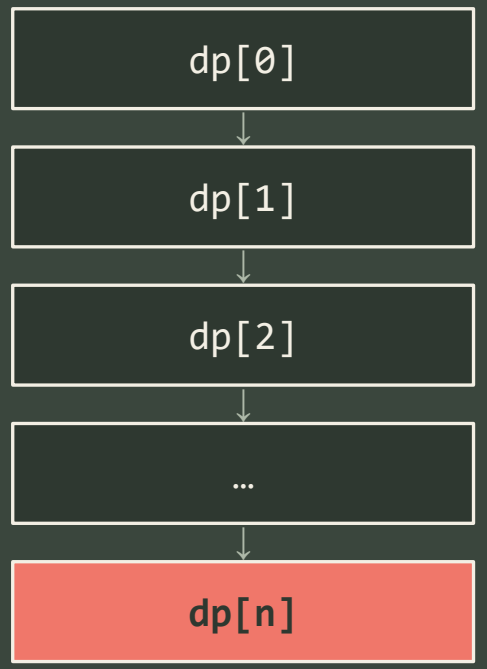
Pattern-first learning: *recognise the shape of the problem before writing any code.*

The DP Framework (Mandatory)

DEFINE THESE FIVE BEFORE YOU CODE

1. **State** dp[i] = the answer for the sub-problem ending at i
2. **Transition** dp[i] = f(dp[i-1], dp[i-2], …)
3. **Base** dp[0], dp[1] — the smallest known answers
4. **Order** fill Left → Right so inputs exist first
5. **Answer** dp[n] — read the final result

The dp[ ] table



fills top → bottom, dp[n] is the answer

Brute Force → DP Flow

HOW WE ARRIVE AT THE DP SOLUTION

1 Recursion

- Try all possibilities
- Exponential time

→

2 Memoization

- Cache repeated results
- Top-down

→

3 DP (Tabulation)

- Build iteratively
- Bottom-up

Core philosophy

Derive, don’t memorize. **Thinking > coding.**

Key Takeaways & Common Mistakes

LOCK IT IN

✓ Key Takeaways

- DP = state + transition + accumulation
- Define the meaning first, not the formula
- Build every answer from smaller problems

! Common Mistakes

- Writing a formula with no dp meaning
- Missing or wrong base case
- Mixing count logic with min/max logic
- Staying in a recursion mindset inside DP

ONE-LINE SUMMARY

Dynamic Programming = building answers using previous answers, systematically.

Derive, don’t memorize · Define meaning first · Build from smaller problems

End of Day 3 — Next up: Take / Skip DP